PassExamAI

Software Design Document (SDD) / Technical Design Document (TDD)

Version 2.0 — MVP Architecture

Competition	Global Competition on Design for Futures (GCD4F) 2026
Theme	AI for Education
Target SDG	SDG 4 — Quality Education
Submission deadline	April 19, 2026
Document version	2.0 — Revised Architecture
Authors	PassExamAI Engineering Team
Coordinator	Eric Toumpe (eric.toumpe@enspy-uy1.cm)

1. Purpose and Scope

This document describes the complete software and technical design of the PassExamAI MVP. It serves as the reference architecture for the development team and as supporting documentation for the GCD4F 2026 competition submission.

PassExamAI is an AI-powered exam preparation platform that transforms uploaded study materials into structured learning roadmaps, interactive lessons, contextual exercises, and mock exams with automated grading.

The system is optimized for:

- A 2–3 person team working in parallel on separate modules
- A one-month delivery deadline targeting a live competition demo
- A distributed two-service architecture (Next.js frontend + FastAPI backend)
- A Smart RAG pipeline combining local document knowledge and live web search
- Production-grade simplicity: no over-engineering, every component earns its place

2. Product Summary

PassExamAI allows a student to:

1. Upload an exam paper, syllabus, course notes, or reference documents (PDF, text)
2. Receive a structured revision roadmap, enriched by both document analysis and live web research
3. Study chapter by chapter through AI-generated lessons grounded in the uploaded material and curated web sources
4. Interact with a contextual AI tutor that answers questions within the current chapter scope
5. Practice with automatically generated exercises: MCQs, short answers, and structured questions
6. Take a timed mock exam and receive automated scoring with detailed AI feedback
7. Track learning progress across sessions

The system is not a general chatbot. It is a structured, document-grounded, web-enriched exam preparation workflow.

3. Goals and Non-Goals

3.1 Goals

- Convert uploaded documents into a structured, AI-generated revision roadmap
- Ground all AI responses in uploaded documents via Smart RAG
- Enrich roadmap and lesson generation with curated, real-time web content
- Provide a fast, contextual AI tutor augmented with live web search for Q&A
- Support assessment, automated grading, and improvement feedback
- Enable parallel team development through strict module ownership and typed contracts
- Deliver a fully demonstrable MVP within one month

3.2 Non-Goals (MVP)

- Full LMS replacement or teacher dashboard
- Voice interaction (input or output)
- Real-time collaborative study sessions
- Native mobile application
- Advanced exam proctoring or anti-cheating systems
- Permanent vector indexing of crawled web content
- Multi-language interface (English only for MVP)

4. System Architecture

4.1 Architecture Overview

PassExamAI uses a distributed two-service architecture connected through a shared Supabase data layer. The frontend and backend are independently deployable services that communicate via a secure proxy pattern.

Service	Technology	Responsibility
Frontend	Next.js 14 (TypeScript)	UI, authentication, proxy to backend, streaming display
Backend	FastAPI (Python)	Smart RAG, AI orchestration, document processing, LLM calls
Database	Supabase (Postgres + pgvector)	Data persistence, vector storage, file storage, auth tokens
External AI	GROQ, OpenRouter, LiteLLM	LLM inference — speed and quality routing
Web Search	Tavily + Firecrawl	Live web enrichment for roadmap, lessons, and chat Q&A
Document Parsing	LlamaParse	PDF extraction — including scanned documents

4.2 The Proxy Pattern

The browser communicates exclusively with Next.js. Next.js Route Handlers authenticate each request via Supabase Auth, then relay it to FastAPI with the validated JWT token. FastAPI is never exposed directly to the public internet.

Communication flow: Browser → Next.js Route Handler (auth + proxy) → FastAPI → Supabase / AI APIs

Benefit	Description
No CORS	FastAPI operates on a private URL — the browser never sees it
Centralized auth	JWT validation happens once in Next.js before any backend call
Separation of concerns	Frontend handles UI state; backend handles all AI and data logic
Streaming support	FastAPI StreamingResponse passthrough via Next.js to the browser

4.3 Service Responsibilities (Strict Boundary)

Next.js — Only does:

- Rendering all UI pages and components
- Supabase Auth (login, signup, session management)
- Supabase Storage uploads (PDF files uploaded directly from the browser)
- Supabase DB direct reads for non-AI data (user profile, progress display)
- Proxying all AI-related requests to FastAPI via Route Handlers

- Consuming FastAPI streaming responses and rendering them token-by-token

FastAPI — Only does:

- Document ingestion pipeline (LlamaParse → chunking → embedding → pgvector)
- Smart RAG pipeline (query rewriting → vector search → re-ranking → context assembly)
- Web search and content extraction (Tavily + Firecrawl)
- All LLM calls (roadmap, lesson, exercise, exam, grading, chat)
- JWT validation and user context extraction from Supabase tokens
- Background job processing for document ingestion
- Streaming AI responses back to the proxy

5. Technology Stack

5.1 Frontend — Next.js

Technology	Role and Rationale
Next.js 14 (App Router)	Full-stack React framework. Server Components for static pages, Route Handlers for the proxy layer.
TypeScript	Type safety across all frontend modules. Shared type definitions with FastAPI contracts.
Tailwind CSS	Utility-first styling. Fast iteration, consistent design system.
shadcn/ui	Component library copied into the project — no external dependency at runtime.
Vercel AI SDK (client only)	useChat() and useCompletion() hooks to consume FastAPI streaming responses in the UI.
Supabase JS Client	Auth session management and direct DB/Storage access from the browser.

5.2 Backend — FastAPI (Python)

Technology	Role and Rationale
FastAPI + Uvicorn	Async Python web framework. Native async/await for non-blocking LLM and web calls.
LlamaIndex (Python)	Smart RAG pipeline: SentenceSplitter, SemanticSplitter, pgvector integration, query engines.
LlamaParse	Cloud API for PDF parsing — handles scanned documents, tables, and complex layouts. Returns clean Markdown.
LiteLLM	Unified LLM client routing GROQ, OpenRouter, and OpenAI through a single interface with automatic fallback.
Pydantic v2	Structured output validation — equivalent of Zod for Python. All AI outputs validated against schemas.
supabase-py	Python client for Supabase DB operations and Storage access from the backend.
python-jose	JWT validation library — verifies Supabase Auth tokens on every incoming FastAPI request.
Tavily Python SDK	Web search and single-URL content extraction for AI enrichment.
Firecrawl Python SDK	Deep web content extraction for multi-page sources during roadmap and lesson generation.

5.3 LLM Routing Strategy

Different tasks require different trade-offs between speed, cost, and quality. LiteLLM provides a single interface to route each task to the optimal provider:

Task	Provider / Model	Rationale

Tutoring chat (Q&A)	GROQ / Llama 3.3 70B	Sub-second response. Free tier. Ideal for interactive Q&A.
Exercise generation	GROQ / Llama 3.3 70B	Fast, structured outputs. Reliable for MCQ format.
Quick feedback	GROQ / Mixtral 8x7B	Cost-efficient. Good enough for short answer feedback.
Roadmap generation	OpenRouter / Gemini Flash 1.5	High quality structured output. Handles long document summaries.
Lesson generation	OpenRouter / Claude Haiku 3.5	Excellent at structured educational content with source grounding.
Full exam generation	OpenRouter / GPT-4o mini	Reliable structured output across diverse question types.
Deep grading	OpenRouter / Claude Haiku 3.5	Nuanced reasoning for subjective answer evaluation.
Text embeddings	OpenAI / text-embedding-3-small	1536 dimensions. Best semantic accuracy. Used by all RAG retrieval.

6. Smart RAG Pipeline

6.1 What is Smart RAG

Smart RAG is PassExamAI's core knowledge retrieval system. It combines offline RAG (from uploaded documents stored in pgvector) with online enrichment (from live web search). Every AI-generated response is grounded in verifiable sources — reducing hallucinations and improving educational quality.

Smart RAG = Offline RAG (pgvector) + Online RAG (Tavily + Firecrawl) + Query Intelligence (rewriting + re-ranking)

6.2 Offline RAG — Document Ingestion Pipeline

Stage 1: Document Parsing

- User uploads PDF to Supabase Storage via the Next.js frontend
- FastAPI receives the storage URL and launches a BackgroundTask for ingestion
- LlamaParse API receives the PDF and returns clean Markdown
- LlamaParse handles: scanned PDFs (OCR), multi-column layouts, tables, mathematical formulas
- Fallback: if LlamaParse fails, use PyMuPDF for text-only PDFs

Stage 2: Intelligent Chunking

- Primary strategy: SentenceSplitter with chunk_size=512 tokens, chunk_overlap=50 tokens
- The overlap ensures context continuity — a concept split across chunks remains retrievable
- Each chunk is tagged with rich metadata: document_id, page_number, section_title, subject_tag, source_type (exam / syllabus / notes / reference), chapter_hint
- Metadata enables filtered retrieval: search within subject X and chapter Y before similarity ranking

Stage 3: Embedding and Storage

- Each chunk is embedded with OpenAI text-embedding-3-small (1536 dimensions)
- Chunks and embeddings are stored in Supabase Postgres via the pgvector extension
- An HNSW index is created at migration time for fast approximate nearest-neighbor search:
  ```
  CREATE INDEX ON document_chunks USING hnsw (embedding vector_cosine_ops)
  WITH (m=16, ef_construction=64);
  ```
- Cosine similarity is used because it measures semantic direction, not magnitude — ideal for text

6.3 Online RAG — Web Search Integration

Tavily — Search and Single-URL Extraction

- Tavily is an LLM-optimized search API that returns clean, relevant results with source URLs and snippets
- Used for: tutoring chat Q&A enrichment, quick factual verification, source attribution
- Tavily also provides a URL extract endpoint — for single pages, no additional tool is needed
- Web content retrieved by Tavily is injected as ephemeral context into the prompt — never permanently indexed in pgvector

Firecrawl — Deep Content Extraction

- Firecrawl crawls full web pages and converts them to clean Markdown
- Used for: roadmap generation (enriching chapter outlines with structured course content), lesson generation (extracting full explanations from educational resources)
- Applied selectively: only the top 2–3 highest-relevance URLs from Tavily results are crawled
- Crawled content is injected as ephemeral generation context — not stored permanently

Critical design principle: Web content is always EPHEMERAL. It enriches prompts but never pollutes the vector database with potentially stale or inaccurate external content.

Usage Matrix — When to Use What

AI Operation	Web Enrichment Method	Purpose
Roadmap generation	Tavily search → Firecrawl (top 3 URLs)	Enrich chapter structure with real course outlines
Lesson generation	Tavily search → Firecrawl (top 2 URLs)	Add depth, examples, and current references
Tutoring chat Q&A	Tavily search only (snippets)	Answer factual questions beyond the uploaded material
Exercise generation	None — uses offline RAG only	Exercises must be grounded in the uploaded curriculum
Exam generation	None — uses offline RAG only	Exam must reflect the student's specific documents
Grading and feedback	Tavily search (verification only)	Verify factual correctness of student answers

6.4 Query Intelligence

Query Rewriting

- Before any vector search, a lightweight LLM (GROQ / Llama 3.3 70B) reformulates the user's question into an optimal retrieval query
- Example: 'I don't understand this' → 'Definition and worked examples of [concept X] from chapter [Y]'

- This single step improves retrieval recall by 25–40% with minimal latency overhead (~200ms)

Re-ranking (Optional MVP stretch)

- After vector search returns top-10 chunks, Cohere Rerank API re-scores them by relevance to the original question
- The top-3 re-ranked chunks are used as the final retrieval context
- Improves precision significantly, especially for multi-topic documents

Context Budget Management

- Each LLM call has a strict context token budget to avoid exceeding model limits (32k for GROQ, 128k for OpenRouter models)
- Priority order when building the prompt: (1) current question, (2) top-3 RAG chunks, (3) web search snippets, (4) chapter context summary, (5) last 3 conversation turns
- Older conversation history is truncated first; retrieval context is always preserved

7. AI Orchestration Design

7.1 Core Principle

Every AI action must be tied to: (1) a typed Pydantic schema, (2) a source context (RAG or web), (3) a narrow task scope. No AI module is general-purpose.

7.2 AI Modules

Roadmap Generator

- Input: document summaries (offline RAG) + web-enriched chapter references (Tavily + Firecrawl)
- Output: RoadmapSchema — list of chapters with title, objective, estimated importance, order index
- Cache check: hash the document content; if a roadmap exists for this hash, return it from DB
- Provider: OpenRouter / Gemini Flash 1.5 with structured JSON output mode

Lesson Generator

- Input: chapter context + top-3 RAG chunks + Firecrawl content (top 2 relevant URLs)
- Output: LessonSchema — explanation, worked examples, visual aids description, source references
- Provider: OpenRouter / Claude Haiku 3.5

AI Tutor (Chat)

- Input: user question → query rewriting → pgvector search (filtered by chapter) → Tavily snippets
- Output: streaming text response with inline source citations
- Provider: GROQ / Llama 3.3 70B — streaming enabled via LiteLLM stream=True
- Conversation history: last 3 turns injected, older turns dropped to manage context budget

Exercise Generator

- Input: chapter context + offline RAG only (no web enrichment — must reflect uploaded curriculum)
- Output: ExerciseSchema[] — MCQ with 4 options, short answer with rubric, structured question with steps
- Provider: GROQ / Llama 3.3 70B

Exam Generator

- Input: roadmap + chapter importance weights + offline RAG
- Output: ExamSchema — full exam with question distribution across chapters
- Provider: OpenRouter / GPT-4o mini with structured output mode

Grader

- Input: student answer + exercise rubric + RAG context for verification + Tavily factual check
- Output: GradingResult — score (0–100), detailed feedback, correct answer, improvement suggestions
- MCQ: deterministic scoring. Short answer: LLM-based rubric evaluation. Structured: partial scoring per step
- Provider: OpenRouter / Claude Haiku 3.5 for nuanced reasoning

7.3 Pydantic Output Schemas

All AI outputs are validated against strict Pydantic v2 models before being returned to the frontend. If validation fails, the system retries once with an explicit correction instruction to the LLM.

Core schemas: RoadmapSchema, ChapterSchema, LessonSchema, ExerciseSchema, ExamSchema, GradingResultSchema — defined in app/schemas/

8. Backend Architecture (FastAPI)

8.1 Project Structure

Module	Responsibility
app/api/	FastAPI route definitions: documents, roadmap, chapters, exercises, exam
app/core/	Configuration (Pydantic Settings), JWT auth middleware, FastAPI dependencies
app/rag/ingestion.py	Full ingestion pipeline: LlamaParse → chunking → embedding → pgvector storage
app/rag/chunking.py	LlamaIndex SentenceSplitter configuration and execution
app/rag/embeddings.py	OpenAI text-embedding-3-small via LiteLLM embedding()
app/rag/retrieval.py	pgvector HNSW cosine search with metadata filtering
app/rag/query_rewriter.py	LLM-based query reformulation before vector search
app/rag/reranking.py	Cohere Rerank integration for precision improvement (optional)
app/web/tavily_client.py	Tavily search and single-URL extraction
app/web/ firecrawl_client.py	Firecrawl deep content extraction for educational pages
app/ai/llm_client.py	LiteLLM wrapper with routing logic per task type
app/ai/ roadmap_generator.py	Roadmap generation with web enrichment + cache
app/ai/ lesson_generator.py	Lesson generation with offline RAG + Firecrawl enrichment
app/ai/ exercise_generator.py	Exercise generation from offline RAG
app/ai/grader.py	Scoring and feedback with rubric-based evaluation
app/ai/exam_generator.py	Full exam assembly with chapter weighting
app/db/	Supabase client, chunk CRUD, progress tracking, document status updates
app/schemas/	All Pydantic v2 schemas — the typed contracts for AI outputs

8.2 Authentication Design

Every request from Next.js to FastAPI carries a Supabase JWT in the Authorization header. FastAPI validates this token using python-jose and the Supabase JWT secret, extracts the user_id, and injects it as a FastAPI dependency into all route handlers.

- Token validation: python-jose + Supabase JWT secret (from environment variable)
- User context: extracted from JWT claims and injected via FastAPI Depends()
- All DB operations are scoped to the authenticated user_id — row-level isolation

8.3 Asynchronous Document Processing

Document ingestion is time-intensive (parsing, chunking, embedding can take 10–60 seconds). It runs as a FastAPI BackgroundTask — the API responds immediately with 202 Accepted and a job reference.

- The frontend polls GET /api/v1/documents/{id}/status every 3 seconds
- Status transitions: uploaded → parsing → chunking → embedding → ready (or failed)
- On failure: the error message is stored in the document record for display in the UI
- V2 upgrade path: replace BackgroundTasks with a proper job queue (Celery + Redis) if processing volume increases

9. Frontend Architecture (Next.js)

9.1 Project Structure

Path	Content
src/app/(auth)/	Login and signup pages using Supabase Auth
src/app/dashboard/	Project list and creation
src/app/projects/[id]/roadmap/	Roadmap visualization — chapters, objectives, progress indicators
src/app/projects/[id]/chapters/[cid]/	Chapter study page: lesson, tutor chat, exercises
src/app/projects/[id]/exam/	Mock exam interface with timer
src/app/projects/[id]/results/	Exam results and chapter-level feedback
src/app/api/ (Route Handlers)	Proxy layer — one file per FastAPI endpoint group
src/lib/supabase.ts	Supabase client initialization
src/lib/api-client.ts	Typed fetch wrappers for calling Route Handlers
src/components/	Shared UI components: UploadCard, ChapterCard, LessonPanel, ExerciseBlock, ScoreSummary, SourceCitation
src/types/	TypeScript interfaces matching Pydantic schemas from FastAPI

9.2 Proxy Route Handler Pattern

Each Route Handler in src/app/api/ follows the same three-step pattern:

8. Verify the Supabase session and retrieve the JWT access token
9. Forward the request to FastAPI with the JWT in the Authorization header
10. Return the FastAPI response directly to the browser (including streaming responses)

For streaming endpoints (tutoring chat), the Route Handler passes the FastAPI ReadableStream directly to the browser without buffering — ensuring real-time token-by-token display.

9.3 Direct Supabase Operations (No Proxy)

Some operations bypass FastAPI entirely and call Supabase directly from the browser for performance:

- File upload: PDF files are uploaded directly to Supabase Storage from the browser — only the storage URL is sent to FastAPI
- Progress reads: chapter completion status is read directly from Supabase DB
- User profile: account information is managed directly via Supabase Auth client

10. Database Design

10.1 Core Tables

Table	Key Fields	Notes
users	id, email, name, created_at	Managed by Supabase Auth. Referenced by all other tables.
projects	id, user_id, title, subject, target_exam_type	One project per study session / exam preparation
uploaded_documents	id, project_id, filename, storage_url, extracted_text, source_type, status	Status: uploaded → parsing → chunking → embedding → ready
document_chunks	id, document_id, chunk_index, content, embedding (vector), metadata (jsonb)	Core RAG table. embedding column uses pgvector type.
roadmaps	id, project_id, title, status, doc_content_hash	Hash enables caching — same document = same roadmap
chapters	id, roadmap_id, order_index, title, objective, importance, status	status: locked / available / in_progress / completed
lessons	id, chapter_id, content, examples, source_references (jsonb)	Generated once per chapter, stored for reuse
exercises	id, chapter_id, question_type, prompt, expected_answer_schema (jsonb)	question_type: mcq / short_answer / structured
attempts	id, exercise_id, user_id, answer, score, feedback, created_at	One row per student answer submission
mock_exams	id, roadmap_id, title, time_limit, generated_at	Assembled from chapters with importance weighting
exam_questions	id, mock_exam_id, chapter_id, question_type, prompt, rubric (jsonb)	Includes the grading rubric for AI scoring
exam_submissions	id, mock_exam_id, user_id, total_score, section_scores (jsonb), feedback	One per exam attempt
progress	id, user_id, chapter_id, completion_status, last_seen_at	Drives the progress indicators in the roadmap view

10.2 Vector Storage and Indexing

- The document_chunks table uses the pgvector extension column type: embedding vector(1536)
- HNSW index for fast approximate nearest-neighbor search (created at migration time):
  ```
  CREATE INDEX ON document_chunks USING hnsw (embedding vector_cosine_ops)
  WITH (m=16, ef_construction=64);
  ```
- Metadata filtering with jsonb operators allows pre-filtering by subject, chapter, or source_type before similarity search — dramatically reducing search space
- Typical query: filter WHERE metadata->>'project_id' = $1 AND metadata->>'chapter_hint' = $2, then ORDER BY embedding <=> $query_embedding LIMIT 10

11. API Contracts (FastAPI Endpoints)

All endpoints are prefixed with /api/v1. All endpoints require Authorization: Bearer <supabase_jwt> header.

11.1 Documents

Method + Path	Request	Response
POST /documents/ingest	{ storage_url, filename, project_id, source_type }	202 { job_id, document_id }
GET /documents/{id}/status	—	{ status, chunks_count, error? }
GET /documents	Query: project_id	{ documents: DocumentSchema[] }
DELETE /documents/{id}	—	204 No Content

11.2 Roadmap

Method + Path	Request	Response
POST /roadmap/generate	{ project_id }	RoadmapSchema with chapters[]
GET /roadmap/{id}	—	RoadmapSchema

11.3 Chapters and Lessons

Method + Path	Request	Response
POST /chapters/{id}/lesson	{ use_web_enrichment: bool }	LessonSchema
POST /chapters/{id}/chat	{ message, history[], chapter_context }	StreamingResponse (text/event-stream)
POST /chapters/{id}/complete	—	{ status: 'completed' }
POST /chapters/{id}/exercises	{ count, types[] }	ExerciseSchema[]
POST /exercises/{id}/grade	{ answer }	GradingResult

11.4 Exam

Method + Path	Request	Response
POST /exam/generate	{ roadmap_id, time_limit?, question_count? }	ExamSchema
GET /exam/{id}	—	ExamSchema
POST /exam/{id}/submit	{ answers: [{question_id, answer}] }	ExamResult with section_scores

11.5 Progress

Method + Path	Request	Response
GET /progress	Query: project_id	{ chapters: ProgressSchema[] }

12. Security Design

Concern	Mitigation
Authentication	Supabase JWT validated on every FastAPI request via python-jose. No unauthenticated endpoint exists.
File access	PDF files stored in Supabase Storage private bucket — only accessible via signed URLs generated server-side.
File validation	File type and size validated on upload. Only PDF and plain text accepted. Max size enforced by Supabase Storage policy.
Secrets management	All API keys stored in environment variables — never in code or version control. Separate .env files per service.
Data isolation	All DB queries include user_id scoping. Row-level security policies enforced at the Supabase level.
Web content safety	Tavily and Firecrawl content is injected as prompt context with explicit framing. Never executed or stored as trusted data.
LLM output validation	All AI outputs validated through Pydantic v2 schemas before use. Invalid outputs are caught and handled gracefully.
Personal data	Minimum data collection. No sensitive personal data sent to external LLM providers beyond what is strictly necessary for the task.

13. Team Collaboration Model

13.1 Module Ownership

Team Member	Owns	Produces
Developer A (Frontend)	All Next.js pages, components, Route Handler proxies, UI state management	Working UI for all screens: upload, roadmap, chapter, exam, results
Developer B (Backend/AI)	All FastAPI modules: RAG pipeline, web search, AI orchestration, DB access, Pydantic schemas	Working API for all endpoints. Streaming responses. Validated AI outputs.
(Optional) Developer C	Prompt engineering, evaluation, corpus preparation, demo content	Optimized prompts, test documents, demo script

13.2 Collaboration Rules

- Every FastAPI endpoint is documented with its request/response schema before any frontend code references it
- Pydantic schemas in app/schemas/ are the single source of truth — TypeScript types in the frontend are derived from them
- No developer modifies another module's internal logic without explicit agreement
- The proxy Route Handlers are the integration boundary — frontend calls them, backend serves them
- All streaming endpoints are tested with a curl call before frontend integration

13.3 Shared Contracts (TypeScript ↔ Python)

Because the frontend is TypeScript and the backend is Python, the two teams share contracts through documentation rather than generated code. For each Pydantic schema on the Python side, a matching TypeScript interface is maintained in src/types/.

14. Development Milestones

Week	Backend (FastAPI)	Frontend (Next.js)
Week 1	Project setup, Supabase schema + HNSW migration, auth middleware, document upload endpoint, LlamaParse integration	Project setup, Supabase auth (login/signup), dashboard UI, document upload form with polling
Week 2	Chunking pipeline, embeddings, pgvector storage, Tavily + Firecrawl clients, query rewriting, roadmap generator with web enrichment	Roadmap display page, chapter list, progress indicators, skeleton loaders
Week 3	Lesson generator (RAG + web), AI tutor streaming, Tavily chat enrichment, exercise generator and grader	Chapter study page, streaming chat UI, exercise interface, answer submission and feedback display
Week 4	Exam generator, exam grading with full scoring, bug fixing, performance tuning, HNSW index verification, demo preparation	Exam interface with timer, results page, progress tracking, UI polish, demo rehearsal

15. Testing Strategy

15.1 Backend Unit Tests

- Chunking utilities: verify chunk sizes, overlap correctness, metadata tagging
- Pydantic schema validation: test that valid and invalid AI outputs are handled correctly
- Query rewriting: verify that reformulated queries are semantically meaningful
- Retrieval functions: test vector search returns correctly ranked results for known queries
- Grading utilities: verify scoring logic for MCQ and rubric-based answers

15.2 Integration Tests

- Upload → LlamaParse → chunking → embedding → pgvector storage (full ingestion pipeline)
- Vector search → lesson generation → Pydantic validation (RAG to lesson flow)
- Tavily search → Firecrawl extraction → roadmap generation (web enrichment flow)
- Exercise submission → grader → GradingResult schema (assessment pipeline)

15.3 Manual Demo Test Script

Before any demo or submission, the following sequence must pass completely:

11. Upload a real exam PDF (e.g. a GCE Advanced Level past paper)
12. Confirm document status reaches 'ready' within 90 seconds
13. Generate the roadmap — verify chapters are coherent and web-enriched
14. Open chapter 1 — generate lesson — verify RAG source references are shown
15. Ask the tutor a question — verify streaming response with web sources
16. Complete exercises for chapter 1 — verify grading feedback is meaningful
17. Generate mock exam — verify question distribution across chapters
18. Submit exam answers — verify total score and section scores are displayed
19. Check progress dashboard — verify chapter completion is reflected

16. Key Risks and Mitigations

Risk	Impact	Mitigation
LlamaParse API down or slow	Ingestion pipeline blocked	Fallback to PyMuPDF for text PDFs. Status page shows clear error.
Tavily or Firecrawl unavailable	Web enrichment disabled	System falls back gracefully to offline RAG only. Lesson quality degrades but pipeline continues.
GROQ rate limit hit during demo	Tutor chat fails	Exponential backoff implemented. OpenRouter as fallback provider in LiteLLM config.
RAG retrieval quality too low	Irrelevant lesson content	Improve chunk metadata tagging. Adjust chunk size. Add re-ranking.
Pydantic validation failure on AI output	Endpoint returns 500	Retry once with correction prompt. Fallback to partial structured response.
pgvector HNSW index missing	Retrieval too slow for demo	Index created at migration time. Verified with EXPLAIN ANALYZE before demo.
Team overload approaching deadline	Features incomplete	Strict scope control. Mock exam is the last feature added — demo flow is the priority.
Internet connectivity during demo	API calls fail	Full local fallback: FastAPI on localhost + ngrok as backup. Pre-generate demo content.

17. MVP Definition of Done

The MVP is complete and ready for demonstration when ALL of the following criteria are met:

20. A user can create an account and log in via Supabase Auth
21. A user can upload a PDF exam paper or course notes
22. The document is parsed by LlamaParse and ingested into pgvector within 2 minutes
23. A roadmap is generated with coherent chapters, enriched by Tavily + Firecrawl web sources
24. A user can study at least one chapter with a generated lesson that cites its sources
25. The AI tutor answers questions in context, streaming responses with web search enrichment
26. A user can complete exercises with automated grading and meaningful feedback
27. A user can generate and take a full mock exam with a timer
28. Exam results display total score, section scores, and improvement guidance
29. Progress is tracked and visible across the roadmap
30. All AI responses cite their sources (RAG chunks or web URLs)
31. The full demo flow completes without errors on a live internet connection

18. Architectural Principles

Principle 1 — Every AI action must be tied to a typed contract, a source context, and a narrow task scope.

Principle 2 — Web content is always ephemeral. It enriches prompts but never pollutes the vector database.

Principle 3 — The proxy boundary is sacred. The browser never calls FastAPI directly. FastAPI never calls Supabase Auth directly.

Principle 4 — MVP first. Every feature decision is evaluated against the demo flow, not the ideal product.

Principle 5 — Sources are non-negotiable. Every AI-generated piece of content must be traceable to either a document chunk or a web URL.

These five principles resolve any ambiguous design decision during development. When in doubt: narrow the task, cite the source, keep the MVP scope.

— End of Document —